Qwen3-TTS CustomVoice

用小张量看模型计算

范围：Qwen3TTSTalker，主干 28 层/2048 维，码本预测器 5 层/1024 维，每帧 16 个码本(Codebook)。

约定：各节数值独立，默认两个 token、二维向量、单头、TP=1；省略 bias 和 eps，数值保留三位。真实模型部分投影有 bias；× 表示逐元素乘，@ 表示矩阵乘。

源码：模型定义、注意力、投影与 MLP。

① model.text_embedding / codec_embedding：嵌入查表(Embedding)

```
input_ids = [0,2] shape=[token数=2]
embedding.weight = [[1,0],[0,1],[2,3]] shape=[词表大小=3, hidden_size=2]
第一步：按 ID 取权重表对应行，hidden_states = weight[input_ids] = [[1,0],[2,3]], shape=[2,2]
第二步：文本向量送入 text_projection；codec_embedding 同理查音频、语言、说话人等 ID。
```

提示(Prompt)会按位置拼接或相加这些条件向量；CustomVoice 用说话人 ID 查表，不调用 speaker_encoder。

② text_projection / ResizeMLP：线性投影(Linear) → SiLU → 线性投影

```
x = [[1,2],[2,1]] shape=[token数=2, 输入维度=2]
W1 = [[1,0],[0,1]]; W2 = [[1,1],[0,1]] 均为 [输出维度=2, 输入维度=2]
第一步：h = x @ W1.T = [[1,2],[2,1]], shape=[2,2]
第二步：h = SiLU(h) = h / (1+exp(-h)) ≈ [[0.731,1.762],[1.762,0.731]]
第三步：output = h @ W2.T ≈ [[2.493,1.762],[2.493,0.731]], shape=[2,2]
```

③ input_layernorm / post_attention_layernorm / model.norm：残差与均方根归一化(RMSNorm)

每个 DecoderLayer：输入归一化 → 注意力 → 残差相加与归一化 → MLP；MLP 的残差相加延后到下一层或最终 norm。

```
hidden_states = [[1,2],[2,1]]; residual = [[2,2],[2,2]] 均为 [token数=2, hidden_size=2]
weight = [2,1] shape=[hidden_size=2]
第一步：x = hidden_states + residual = [[3,4],[4,3]]; residual=None 时省略相加。
第二步：z = x / sqrt(mean(x², 按行)) ≈ [[0.849,1.131],[1.131,0.849]]
第三步：output = z × weight ≈ [[1.697,1.131],[2.263,0.849]], shape=[2,2]; 这是逐维缩放，不是线性投影。
```

④ self_attn.qkv_proj / q_norm / k_norm / rotary_emb：准备查询、键、值(Q/K/V)

x = [[1,0],[0,1]]，shape=[token数=2, hidden_size=2]；Wq=Wk=[[1,0],[0,1]]；Wv=[[2,0],[0,2]]，权重均为 [输出维度=2, 输入维度=2]。

第一步：Q=x@Wq.T=[[1,0],[0,1]]；K=Q；V=x@Wv.T=[[2,0],[0,2]]，shape=[token数=2, head_dim=2]

第二步：Q/K 分别做 RMSNorm，weight=[1,1]，得到 Q=K≈[[1.414,0],[0,1.414]]；V 不变。

第三步：旋转位置编码(RoPE)只旋转 Q/K；示意角度 [0°,90°]，旋转后 Q=K≈[[1.414,0],[-1.414,0]]。

旋转公式：[a,b] → [a*cosθ-b*sinθ, a*sinθ+b*cosθ]；实际角度来自位置和频率，形状仍为 [2,2]。

⑤ self_attn.attn / o_proj：因果注意力(Causal Attention)与输出投影

本节独立给定已处理的 Q/K/V，不接上一节的数值；第一个 token 不能看第二个 token。

Q=K=[[√2,0],[0,√2]]；V=[[2,0],[0,2]] shape=[token数=2, head_dim=2]

第一步：scores = Q @ K.T / √2 + mask = [[√2,-∞],[0,√2]]，shape=[查询token数=2, 键token数=2]

第二步：P = softmax(scores, 按行) ≈ [[1,0],[0.196,0.804]]；A=P@V≈[[2,0],[0.391,1.609]]

第三步：Wo=[[1,1],[0,1]]，shape=[输出维度=2, 各头拼接维度=2]；output=A@Wo.T≈[[2,0],[2,1.609]]。

多头时先拼接各头结果再做 o_proj；解码(Decode)时从 GPU 显存(GPU memory)中的 K/V 缓存(KV cache)读取历史键和值。

⑥ mlp / Qwen3OmniMoeTalkerDenseMLP：门控前馈网络(SwiGLU)

x=[[1,2],[2,1]]，shape=[token数=2, hidden_size=2]；Wgate=Wup=[[1,0],[0,1]]，shape=[中间维度=2, 输入维度=2]

第一步：gate=x@Wgate.T=x；up=x@Wup.T=x，两者 shape=[2,2]。

第二步：h=SiLU(gate) × up ≈ [[0.731,3.523],[3.523,0.731]]，shape=[2,2]

第三步：Wdown=[[1,1],[0,1]]，shape=[输出维度=2, 中间维度=2]；output=h@Wdown.T≈[[4.254,3.523],[4.254,0.731]]。

⑦ model.norm → codec_head → 采样(Sampling)：得到第 0 码本 token

28 层之后做③的最终归一化；预填充(Prefill)只选每个请求最后位置的状态做预测。下面两行代表两个请求本轮的状态。

hidden_states=[[1,2],[2,1]]，shape=[请求数B=2, hidden_size=2]；W=[[1,0],[0,1],[1,1]]，shape=[简化词表=3, hidden_size=2]

第一步：logits=hidden_states@W.T=[[1,2,3],[2,1,3]]，shape=[B=2, 词表=3]

第二步：probs=softmax(logits)≈[[0.090,0.245,0.665],[0.245,0.090,0.665]]。

第三步：示例用 argmax 得到 layer0_codes=[2,2]，shape=[B=2]；实际采样还受温度、top-k/top-p、禁用 token 等影响。

⑧ code_predictor：同一帧内依次预测其余码本

真实输入投影 small_to_mtp_projection 为 2048→1024；共享同一套 5 层 Transformer，用 15 个 lm_head 依次预测码本 1～15。

以下只看一个请求的一帧，把 16 个码本简化为 3 个；layer0_code=0，talker_hidden=[1,2]，E0(0)=[3,4]。

第一步：按序列维拼接 pair=[[1,2],[3,4]]，shape=[前缀token数=2, hidden_size=2]；投影 W=[[1,1]]，shape=[1,2]，pair@W.T=[[3],[7]]。

第二步：pair 经过共享 Transformer，取第二个 token 的输出送入 lm_head[0]；略去内部计算，假设 logits=[1,3,2]，argmax 得到 code1=1。

第三步：code1 查对应 embedding 表并投影 → 共享 Transformer → lm_head[1]；假设 logits=[1,2,3]，得到 code2=2；最终 codes=[0,1,2]，真实每帧为 [16]。

⑨ output_embeds / decode_feedback_embedding：准备下一帧输入

仅沿用 codes=[0,1,2]；三组 embedding 表另设为 E=[[1,0],[0,1],[2,2]]，shape=[词表=3, hidden_size=2]。

第一步：feedback=E0[0]+E1[1]+E2[2]=[1,0]+[0,1]+[2,2]=[3,3]，shape=[hidden_size=2]。

第二步：下一份文本向量 text=[0.5,0]，input_embed=feedback+text=[3.5,3]；文本耗尽时改加 tts_pad 向量。

第三步：将 input_embed 写入 GPU 显存(GPU memory)中 decode_feedback_embedding 的请求槽位，下轮按槽位取出；这张表用于暂存反馈。

⑩ vocoder 阶段：音频解码器(Audio Decoder)把编码还原为波形

此步骤在 Talker 外执行，简化为接口：audio_codes [帧数T, 16] → tokenizer.decode(...) → waveform [采样点数N] + sample_rate=24000。

完整链路：文本/条件向量 → Talker 预测首码本 → Code Predictor 补齐一帧 → 反馈并循环 → 声码器(Vocoder)输出声音。